DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

COMPANION OBJECT

- Para criarmos classes estáticas em Kotlin devemos utilizar o companion object. O comando **static** não existe em Kotlin.
- Classes estáticas podem ser acessadas sem serem instanciadas, ou seja, é permitido acessar métodos ou atributos diretamente da classe sem criar um objeto dessa classe.
- Vamos criar um exemplo:

COMPANION OBJECT

```
class Formula {
    fun media( val1: Double, val2: Double): Double {
        return (val1 + val2) / 2.0
    }
}

fun main() {
    val objMedia: Formula = Formula()

    val resultado = objMedia.media( val1: 6.7,  val2: 4.3)
}
```

```
class Formula {
    fun media( val1: Double, val2: Double): Double {
        return (val1 + val2) / 2.0
    }
}

fun main() {
    val resultado = Formula.media( 6.7, 4.3)
    val resultado2 = media( 9.0, 5.8)
}
```

Nos dois casos, o método media fica em vermelho indicando erro.

- Criamos uma classe Formula que tem um método media (calcula a media)
- Para acessarmos esse método é obrigatório instanciar a classe gerando um objeto (objMedia).

COMPANION OBJECT

- Na verdade ao instanciar essa classe apenas para acessar esse método, estamos criando um objeto na memória que além de não guardar nenhum dado sobre o próprio objeto, só é acessado uma ou algumas vezes.
- Tecnicamente não precisamos instanciar o mesmo, só precisamos poder acessar o mesmo e receber o resultado.
- Para esses casos, podemos utilizar métodos estáticos, que no Kotlin temos que declarar os mesmos dentro de um companion object:

COMPANION OBJECT

```
class Formula {
    companion object {
        fun media(val1: Double, val2: Double): Double {
            return (val1 + val2) / 2.0
        }
    }
}

fun main() {
    val resultado = Formula.media( val1: 6.7,  val2: 4.3)

}
```

```
class Formula {
    companion object {
        fun media(val1: Double, val2: Double): Double {
            return (val1 + val2) / 2.0
        }
    }
}

fun main() {
    val resultado = Formula.media( val1: 6.7,  val2: 4.3)

    val teste: Formula = Formula()
    teste.media( 5.0, 4.0)
}
```

- Agora podemos acessar o método media sem instanciar a classe Formula.
- A classe pode ser instanciada, mas pelo objeto os métodos e atributos estáticos não ficam acessíveis.

COMPANION OBJECT

- O companion object pode ser nomeado, mas isso é opcional.

```
class Formula {
    companion object Calculos {
        fun media(val1: Double, val2: Double): Double {
            return (val1 + val2) / 2.0  }
    }
}

fun main() {
    val resultado = Formula.Calculos.media( val1: 6.7,  val2: 4.3)
}
```

- Só é permitido criar um companion object na classe.

```
class Formula {
    companion object Calculos {
        fun media(val1: Double, val2: Double): Double {
            return (val1 + val2) / 2.0  }
    }
    companion object Teste {
        val ttt: Int = 0
    }
}
```

OBJECT

- Em alguns casos o desenvolvedor quer subdividir seus métodos/atributos estáticos, criando nomes para cada conjunto, para isso podemos utilizar o object no lugar do companion object.
- Podemos criar quantos object desejarmos na mesma classe, mas é obrigatório colocar nome em cada um deles
- Para acessar seus métodos é obrigatório utilizar o nome do object.

OBJECT

```
class Formula {
    object Calculos {
        fun media(val1: Double, val2: Double): Double {
            return (val1 + val2) / 2.0
        }
        fun soma(val1: Double, val2: Double): Double {
            return val1 + val2
        }
    }
    object Reajustes {
        val janeiro: Double = 0.9
        val fevereiro: Double = 1.2
        val marco: Double = 1.0
    }
}

fun main() {
    val resultado = Formula.Calculos.media( val1: 6.7, val2: 4.3)

    val reajusteJaneiro = Formula.Reajustes.janeiro
}
```

CLASSE ABSTRATA

- Uma classe que nunca vai ser instanciada (nem acessada de forma estática), só vai servir de modelo (herança) para outras classes.
- Por exemplo: temos as classes geladeira, fogão e micro-ondas, todas herdam a classe eletrodoméstico.
- A classe eletrodoméstico tem métodos e atributos genéricos que existem em todas as sub-classes listadas acima, mas ela nunca vai ser instanciada sozinha, essa classe pode ser definida como abstrata.
- Uma classe abstrata é um modelo para as classes filhas.

CLASSE ABSTRATA

- Usando o modelo de funcionários que são professores ou administrativos utilizado na aula anterior, podemos declarar a classe Funcionario como sendo abstrata utilizando a palavra abstract antes da palavra class:

```
abstract class Funcionario() {
    var matricula: Int = 0
        set(value) {
            if (value>0) field = value
        }
    var nome: String = ""
```

- Agora não é mais permitido instanciar a classe Funcionario.

CLASSE ABSTRATA

- Uma classe sendo abstrata ou não pode conter métodos abstratos.
- Um método abstrato é um método que vai ser diferente em cada uma das sub-classes, mas tem sua criação obrigatória.
- Imagine que temos um método para gerar um bônus para os funcionários, mas esse bônus é sempre de 10% para os professores e de 12% para os administrativos.
- Criamos o método bonus na classe Funcionario indicando que o mesmo é abstrato, isso faz com que seja obrigatório criar os mesmo nas sub-classes (Professores e Administrativos)

CLASSE ABSTRATA

```
abstract class Funcionario() {
    var matricula: Int = 0
        set(value) {
            if (value>0) field = value
        }
    var nome: String = ""
        set(value){
            if (value != "") field = value
        }
    var salario: Double = 0.0
        set(value){
            if (value>0) field = value
        }

    constructor(mat:Int, nome:String, sal:Double): this(){
        matricula = mat
        this.nome = nome
        salario = sal
    }

    abstract fun bonus(): Double

    fun aumentoSalario( porcentagem: Double){
        if (porcentagem > 0) {
            val valor = salario * porcentagem / 100
```

- Agora é obrigatório criar o método bonus nas sub-classes

```
class Administrativo: Funcionario {
    var departamento: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Departamento não cadastrado"
        }
    constructor(mat:Int, nome:String, sal:Double, departamento: String): supe
        this.departamento = departamento
    }

    override fun toString(): String {
        return super.toString() +
                "\nDepartamento: $departamento"
    }
}
```

CLASSE ABSTRATA

```
class Administrativo: Funcionario {
    var departamento: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Departamento não cadastra
        }
    constructor(mat:Int, nome:String, sal:Double, departamento: String):
        this.departamento = departamento
    }

    override fun bonus(): Double {
        return salario * 0.12
    }

    override fun toString(): String {
        return super.toString() +
                "\nDepartamento: $departamento"
    }
}
```

```
class Professor: Funcionario {
    var curso: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Curso não cadastra
        }

    constructor(mat:Int, nome:String, sal:Double, curso: String):
        this.curso = curso

    }

    override fun bonus(): Double {
        return salario * 0.10
    }

    override fun toString(): String {
        return super.toString() +
                "\nCurso....: $curso"
    }
}
```